

Lab 01 — Containers en architectuur: Docker, Podman en de Linux-kernel

Theorie — wat een container werkelijk is, wie wat doet wanneer je `podman run` typt, en waarom "Docker" vandaag zowel een tool als een manier van werken betekent.

Doelstellingen

- Kunnen zeggen wat een container is **zonder** de woorden "lichte virtuele machine" te gebruiken.
- De twee mechanismen van de Linux-kernel benoemen die containers mogelijk maken.
- De actoren uit elkaar houden: client, engine (met of zonder daemon), image, registry — bij Docker **én** bij Podman.
- Een **image** onderscheiden van een **container** — de duurste verwarring voor een beginner.
- Eender welk `docker` / `podman` -commando lezen en de structuur ervan raden.

1. Een verhaal over "bij mij werkt het"

Een applicatie draait nooit alleen. Een Spring Boot-API heeft een JRE in een precieze versie nodig, omgevingsvariabelen, een certificaat, een tijdzone. Dat geheel heet de **runtime-omgeving**, en twintig jaar lang werd ze met de hand op servers geïnstalleerd. Gevolg: de werkpost van de ontwikkelaar en productie waren nooit identiek, en twee applicaties op dezelfde server vochten om dezelfde bibliotheek in twee versies.

Het eerste antwoord was de **virtuele machine**: een volledige gesimuleerde computer, met een eigen besturingssysteem, per applicatie. Het werkt — ten koste van meerdere GB schijf, gereserveerd RAM en een opstart in minuten, om *één* proces te draaien.

→ GESCHIEDENIS

In maart 2013 stelt Solomon Hykes Docker in vijf minuten voor op de PyCon-conferentie. Technisch is er niets nieuws: *namespaces* en *cgroups* zitten sinds 2008 in Linux, LXC gebruikt ze al. Wat Docker uitvindt, is de **verpakking**: een image die je bouwt, publiceert en met één commando start. In 2015 draagt Docker het image-formaat en de runtime over aan het **OCI** (*Open Container Initiative*): sindsdien is een image een standaard die elke tool kan draaien. Podman (Red Hat, 2018) is uit die opening geboren.

De container is het tweede antwoord: **isoleer het proces, niet de machine**.

2. Wat een container is

ONTHOUDEN

Een container is een **gewoon proces** op je Linux-machine, waartegen de kernel liegt over wat het kan zien en wat het mag verbruiken.

Er zit geen besturingssysteem in een container. Geen emulatie. Als je een `nginx` -container start, bestaat er op je host een echt `nginx` -proces, zichtbaar in `ps aux`, uitgevoerd door **dezelfde Linux-kernel** als al de rest. Wat verandert, is wat dat proces van de wereld waarneemt. Twee kernelmechanismen doen dat werk.

→ LINUX

De **kernel** is het deel van het systeem dat met de hardware praat en scheidsrechter speelt tussen programma's: hij maakt processen aan, geeft ze geheugen, CPU-tijd, toegang tot bestanden en netwerk. Alles wat "erboven" zit — `bash`, `ls`, `java`, `nginx` — heet **userland**. Een programma raakt de hardware nooit rechtstreeks aan: het vraagt het aan de kernel via **systeemaanroepen** (*system calls*: `open`, `read`, `fork` ...). Die grens is wat containers benutten.

Namespaces — de isolatie van het zicht. De kernel geeft een proces een gedeeltelijk, privé zicht op bepaalde resources:

Namespace	Wat hij isoleert	Zichtbaar gevolg
<code>pid</code>	De procesidentificaties	In de container is je applicatie PID 1 en ziet ze niets anders
<code>net</code>	Interfaces, poorten, routes	De container heeft zijn eigen poort 8080, los van die van de host
<code>mnt</code>	De mountpunten	De container ziet zijn eigen <code>/</code>
<code>uts</code>	De hostnaam	<code>hostname</code> geeft de container-ID terug
<code>ipc</code>	Interprocescommunicatie	Geen gedeeld geheugen met de burens
<code>user</code>	De UID's/GID's	Een <code>root</code> in de container kan op de host een gewone gebruiker zijn — dit is de kern van rootless Podman

Cgroups (control groups) — de beperking van resources. Namespaces verbergen, cgroups begrenzen: "deze groep processen krijgt niet meer dan 512 MB RAM en 1,5 core". Dat is wat verhindert dat een op hol geslagen container de server meesleurt.

Container of VM?

	Virtuele machine	Container
Isoleert	Een hele computer (gevirtualiseerde hardware)	Eén of meer processen
Kernel	Een eigen kernel	Die van de host, gedeeld
Opstart	Seconden tot minuten	Milliseconden
Typisch gewicht	Meerdere GB	Enkele tientallen MB
Veiligheidsgrens	Sterk (hypervisor)	Zwakker (één kernel te compromitteren)

→ WINDOWS / WSL

"De container deelt de kernel van de host" heeft een gevolg: een Linux-container draait **alleen** op een Linux-kernel. Op Windows levert **WSL 2** (*Windows Subsystem for Linux*) die: een heel lichte VM, beheerd door Hyper-V, die in één seconde opstart, RAM deelt met Windows en een echte Linux-kernel draait die Microsoft compileert. Je Podman draait *in* die Ubuntu-distributie; je containers zijn processen van die VM, niet van Windows. Docker Desktop en Podman Desktop doen achter de schermen hetzelfde: ze maken hun eigen WSL-distributie aan.

3. Image en container

Dit is het fundamentele onderscheid van de hele opleiding.

Een **image** is een **alleen-lezen** sjabloon: een bevroren bestandssysteem (de JRE, je JAR, de bibliotheken) plus metadata (welk commando starten, welke variabelen, welke gebruiker, welke poort). Een image voert niets uit, verbruikt geen CPU, "draait" niet. Ze is inert en **onveranderlijk**.

Een **container** is een *draaiende instantie* van een image: de image, plus een dunne schrijflaag eigen aan die instantie, plus een levend proces.

→ JAVA

De juiste analogie komt uit objectgeoriënteerd programmeren: de image is de **klasse**, de container is het **object** (`new`). Je instantieert twintig containers van dezelfde image; ze delen dezelfde alleen-lezen inhoud en hebben elk hun eigen privétoestand. En net als een object wordt een container vernietigd zonder dat de klasse beweegt.

ONTHOUDEN

Alles wat je applicatie in een container schrijft, gaat in die schrijflaag, die **samen met de container vernietigd wordt**. Dat is de bedoeling: een container is wegwerpbaar. Persistentie is het onderwerp van lab 06.

Een image bestaat uit gestapelde **lagen** (*layers*), één per bouwstap; tien images op basis van dezelfde `eclipse-temurin:21-jre` bewaren die basis maar één keer (lab 02).

4. De architectuur: wie doet wat

Hier lopen Docker en Podman uiteen — en dat is de bestaansreden van Podman.

DOCKER	<code>docker</code> (client)	→ HTTP/socket →	<code>dockerd</code> (daemon, root)	→	<code>containerd</code>	→	<code>runc</code>
PODMAN	<code>podman</code> (jouw gebruiker)	→ fork/exec →	<code>conmon</code>	→	<code>crun</code>		(geen daemon)
	beide	→ pull →	Registry (Docker Hub, Harbor, ECR...)				

Docker is een **client/server**-architectuur. Het `docker`-binary doet bijna niets: het vertaalt je commando naar een HTTP-verzoek aan `dockerd`, een permanente **daemon**, die als **root** draait, al het werk doet (bouwen, aanmaken, opslaan) en luistert op een Unix-socket, `/var/run/docker.sock`.

Podman heeft **geen daemon**. Elk `podman`-commando is een gewoon programma dat het werk zelf doet en dan stopt; de container overleeft dankzij een piepkleine toezichthouder, `conmon`, die eraan vast blijft hangen. En vooral: Podman draait standaard **rootless**: het is *jouw* gebruiker die de container start, zonder privileges. De `root` die je in de container zult zien, is een illusie van de `user`-namespace: aan hostzijde ben jij het.

PODMAN

Waarom een tweede tool? Twee verwijten die operationele teams Docker maakten: **een permanente root-daemon** (één enkel faalpunt, en "wie met de socket kan praten, is root") en **een licentie** (Docker Desktop is sinds 2021 betalend in bedrijven). Podman beantwoordt beide: geen daemon, standaard rootless, gratis. En het maakte een beslissende keuze: zijn CLI is **identiek** aan die van Docker. `alias docker=podman` volstaat in 95 % van de gevallen; images, Dockerfiles en registries zijn dezelfde, omdat dat allemaal OCI is. Je leert dus *Docker* — het vocabularium van de werkvloer — met Podman als motor.

De twee delen de rest: de **registry**, de externe opslag van images (standaard Docker Hub; in bedrijven Harbor, Nexus, GitLab Registry, ECR, ACR), en de laagniveau-**runtime** (`runc` of `crun`), die de kernel werkelijk vraagt de namespaces aan te maken. Zijn naam duikt op in foutmeldingen.

Twee praktische gevolgen om meteen te begrijpen:

1. **Bij Docker gebeurt het werk aan de kant van de daemon.** Een pad gemount met `-v /data:/data` wordt opgelost op de schijf van de *daemon*, niet van de client — lokaal onzichtbaar, de bron van de helft van de verrassingen tegenover een daemon op afstand. Bij Podman zijn client en engine hetzelfde proces.
2. **Toegang tot de Docker-daemon = root-toegang op de host.** Wie naar `/var/run/docker.sock` kan schrijven, kan een geprivilegieerde container starten en de machine overnemen.

→ **BEVEILIGING**

Lid zijn van de groep `docker` komt neer op `sudo` zonder wachtwoord en zonder auditspoor. Rootless Podman is het structurele antwoord: een gecompromitteerde container heeft alleen *jouw* rechten.

5. Anatomie van een commando

De CLI volgt een regelmatige grammatica, dezelfde voor beide tools:

```
podman [object] [actie] [opties] [doel] [argumenten]
```

```
podman container run -d --name api -p 8080:8080 docker.io/library/eclipse-temurin:21-jre java -
version
#      |object| |actie| |opties| |image| |
commando |
```

De belangrijkste objecten zijn `image`, `container`, `volume`, `network`, `system` (en `pod`, eigen aan Podman). Voor de meest voorkomende bewerkingen bestaan er **historische verkorte vormen** waarbij het object impliciet is — die lees je overal:

Volledige vorm	Gangbare verkorte vorm
<code>podman container run</code>	<code>podman run</code>
<code>podman container ls</code>	<code>podman ps</code>
<code>podman image ls</code>	<code>podman images</code>
<code>podman image pull</code>	<code>podman pull</code>
<code>podman container rm</code> / <code>podman image rm</code>	<code>podman rm</code> / <code>podman rmi</code>

Drie diagnosecommando's om vanaf nu te kennen:

```
podman version  # versie van de client (en van de server, als die er is)
podman info     # toestand van de engine: rootless?, cgroups, netwerk, opslag, kernel
podman inspect  # alle metadata van een object, als JSON
```

VALKUIL

Bij Docker toont `docker version` twee blokken, *Client* en *Server*; ontbreekt het tweede, dan draait de daemon niet of mag je er niet mee praten — het probleem zit nooit in je commando. Bij Podman is er maar één blok: er is geen server. De melding "Cannot connect to the Docker daemon" die je in elke FAQ vindt, bestaat dus niet... tenzij je `podman --remote` of `podman machine` gebruikt.

6. In het bedrijf

Op een Spring Boot + Angular + PostgreSQL-stack:

- De Spring Boot-backend wordt **één image** met een JRE en de JAR. Dezelfde image, tot op de **digest**, gaat naar integratie, acceptatie en productie: het einde van "bij mij werkt het".
 - De Angular-frontend wordt *gebuid* (`ng build`) en het statische resultaat wordt in een nginx-image gestopt. Node overleeft de productie niet — lab 05.
 - PostgreSQL komt uit een officiële publieke image; je schrijft ze niet, je configureert ze.
 - Die drie images leven in een **private registry**, gepusht door de CI. Op de servers draaien ze onder Docker, Podman of Kubernetes: de image weet niet wie ze start, en dat is de bedoeling.
-

Onthouden

- Een container is een **geïsoleerd proces** door *namespaces* en beperkt door *cgroups*, uitgevoerd door de **kernel van de host** — geen mini-VM. Op Windows is die kernel die van WSL 2.
- Een **image** is een onveranderlijk sjabloon; een **container** is er een levende instantie van, met een wegwerpbaar schrijflaag.
- Docker = client + permanente root-daemon; Podman = een programma zonder daemon, standaard rootless. Dezelfde CLI, dezelfde images, dezelfde registries (OCI).
- Containers zijn **wegwerpbaar**: elke toestand die niet naar buiten is gebracht, verdwijnt bij het verwijderen.
- Toegang tot de Docker-daemon = root-toegang; rootless Podman geeft alleen jouw rechten.
- De CLI volgt `podman <object> <actie>`; de korte vormen (`ps`, `run`, `images`) zijn verkorte schrijfwijzen.

Woordenschat

image: onveranderlijk sjabloon, stapel alleen-lezen lagen. — **container**: draaiende instantie van een image. — **layer** (laag): fragment van bestandssysteem uit één bouwstap. — **registry**: server die images opslaat en verdeelt. — **repository**: alle versies van eenzelfde image (`postgres`). — **tag**: label van één versie (`postgres:16-alpine`). — **daemon**: permanente dienst (`dockerd`); afwezig bij Podman. — **rootless**: modus waarin engine en containers onder je eigen gebruiker draaien. — **namespace**: isolatie van het zicht op een resource door de kernel. — **cgroup**: verbruiksbeperking. — **runtime**: `runc` / `crun`, de component die de container werkelijk aanmaakt. — **conmon**: kleine toezichthouder van Podman, gekoppeld aan elke container. — **OCI**: open standaard voor images en runtimes.